LAPORAN UJIAN TENGAH SEMESTER
MATA KULIAH MACHINE LEARNING

Klasifikasi Kualitas Air (Water Potability) Menggunakan Decision Tree dengan Perbandingan Metode Resampling SMOTE dan Random Under Sampling (RUS)



Disusun oleh:

KELOMPOK 3 - TI C2

Nabila Risha Juliana	434241017
Adella Putri Asy'ari	434241023
Juanne Meilanie Raya Maturbongs	434241026

PROGRAM STUDI D4 TEKNIK INFORMATIKA
FAKULTAS VOKASI
UNIVERSITAS AIRLANGGA SURABAYA

2026

DAFTAR ISI

DAFTAR ISI **2**
BAB 1
PENDAHULUAN **3**
1.1 Latar Belakang 3
1.2 Deskripsi Dataset 3
1.3 Tools dan Library 4
1.4 Repository GitHub 4
BAB 2
PEMBAGIAN TUGAS **5**
BAB 3
HASIL DAN PEMBAHASAN **6**
3.1 Preprocessing Dataset 6
3.1.1 Kode Program preprocessing.py 6
3.1.2 Penjelasan Kode 7
3.1.3 Output Terminal preprocessing.py 7
3.1.4 Visualisasi Preprocessing 8
3.2 SMOTE + Decision Tree 10
3.2.1 Kode Program smote_pipeline.py 11
3.2.2 Penjelasan Kode 11
3.2.3 Output Terminal smote_pipeline.py 12
3.2.4 Visualisasi SMOTE 12
3.3 RUS + Decision Tree 15
3.3.1 Kode Program rus_pipeline.py 15
3.3.2 Penjelasan Kode 15
3.3.3 Output Terminal rus_pipeline.py 16
3.3.4 Visualisasi RUS 16
3.3.5 Kode Program main.py 18
BAB 4
PERBANDINGAN DAN ANALISIS **19**
4.1 Tabel Perbandingan Metrik 19
4.2 Analisis Confusion Matrix 19
4.3 Interpretasi Hasil 19
4.4 Rekomendasi Metode 20
BAB 5
KESIMPULAN DAN SARAN **21**
5.1 Kesimpulan 21
5.2 Saran 21

BAB 1
PENDAHULUAN

1.1 Latar Belakang

Air merupakan kebutuhan dasar manusia yang secara langsung mempengaruhi kesehatan. Kualitas air minum yang buruk dapat menyebabkan berbagai penyakit, sehingga penting untuk dapat mengklasifikasikan apakah suatu sampel air layak dikonsumsi atau tidak berdasarkan parameter fisik dan kimianya.

Machine Learning menawarkan pendekatan otomatis untuk melakukan klasifikasi tersebut. Namun, dalam kasus data kualitas air yang tersedia secara publik, distribusi kelasnya cenderung tidak seimbang (imbalanced), sehingga model klasifikasi berisiko bias terhadap kelas mayoritas. Untuk mengatasi hal ini, digunakan teknik resampling seperti Synthetic Minority Oversampling Technique (SMOTE) dan Random Under Sampling (RUS).

Laporan ini membandingkan performa dua metode resampling tersebut pada dataset Water Potability menggunakan algoritma klasifikasi Decision Tree, kemudian mengevaluasi hasilnya dengan metrik Confusion Matrix, Accuracy, Precision, Recall, dan F1-Score.

1.2 Deskripsi Dataset

Dataset yang digunakan adalah Water Quality / Potability Dataset yang diunduh dari Kaggle. Dataset ini memiliki karakteristik sebagai berikut:

Keterangan	Detail
Sumber	kaggle.com/datasets/adityakadiwal/water-potability
Jumlah Baris	3.276 baris data
Jumlah Kolom	10 kolom (9 fitur + 1 target)
Tipe Data Fitur	Semua numerik (float)
Target Kelas	Potability (0 = Tidak Layak Minum, 1 = Layak Minum)
Distribusi Kelas	Kelas 0: 1.998 (60,99%) | Kelas 1: 1.278 (39,01%) – IMBALANCED
Missing Values	ph (491), Sulfate (781), Trihalomethanes (162) – total 1.434
Duplikat	Tidak ada

Sembilan fitur numerik yang tersedia adalah: pH, Hardness, Solids, Chloramines, Sulfate, Conductivity, Organic Carbon, Trihalomethanes, dan Turbidity. Kesembilan fitur inilah yang digunakan untuk memprediksi kolom target Potability.

1.3 Tools dan Library

Program dikembangkan menggunakan bahasa Python 3.11 dengan library-library berikut:

Library	Fungsi
pandas	Manipulasi data (load CSV, DataFrame, handle missing value)
numpy	Operasi numerik dan array
scikit-learn	Decision Tree, train_test_split, dan metrik evaluasi
imbalanced-learn	Implementasi SMOTE dan Random Under Sampling
matplotlib	Visualisasi data (bar chart, boxplot, decision tree)
seaborn	Visualisasi statistik (heatmap confusion matrix)

1.4 Repository GitHub

Seluruh kode program dikembangkan secara kolaboratif dan disimpan pada repository GitHub berikut, sehingga riwayat kontribusi masing-masing anggota kelompok dapat ditelusuri melalui log commit:

https://github.com/nabilarishajuliana/UTS-Kel3-ML-WaterPotability

BAB 2
PEMBAGIAN TUGAS

Untuk mempermudah pengerjaan dan tracking kontribusi anggota, tugas dibagi menjadi tiga bagian utama sesuai dengan alur flowchart pada soal. Pembagian tugas dilakukan dengan mempertimbangkan dependensi antar bagian: bagian preprocessing dikerjakan lebih dahulu karena output-nya menjadi input untuk kedua bagian resampling lainnya.

Anggota	File	Bagian yang Dikerjakan
Adella Putri Asy'ari	preprocessing.py	Preprocessing dataset: handle missing value dengan median, deteksi & handle outlier menggunakan metode IQR, Simple Feature Scaling (X/X.max), dan Split data 80% training – 20% testing dengan stratified sampling.
Juanne Meilanie Raya Maturbongs	smote_pipeline.py	Menerapkan SMOTE (oversampling) pada data training, melatih Decision Tree, melakukan prediksi pada data testing, dan mengevaluasi hasilnya menggunakan Confusion Matrix beserta metrik Accuracy, Precision, Recall, dan F1-Score.
Nabila Risha Juliana	rus_pipeline.py	Menerapkan Random Under Sampling (undersampling) pada data training, melatih Decision Tree, melakukan prediksi pada data testing, dan mengevaluasi hasilnya menggunakan Confusion Matrix beserta metrik Accuracy, Precision, Recall, dan F1-Score.

File main.py berperan sebagai penghubung yang memanggil ketiga bagian secara berurutan (Preprocessing → SMOTE Pipeline → RUS Pipeline), sehingga seluruh alur dapat dijalankan sekaligus dengan satu perintah python main.py.

BAB 3
HASIL DAN PEMBAHASAN

Bab ini menjelaskan implementasi kode program dari masing-masing bagian, output yang dihasilkan pada terminal saat program dieksekusi, serta visualisasi berupa grafik yang dihasilkan. Setiap sub-bab merepresentasikan satu bagian pengerjaan dari anggota kelompok.

3.1 Preprocessing Dataset

Bagian ini melakukan pembersihan dan persiapan data mentah menjadi bentuk yang siap dilatih model. Terdapat lima tahapan utama: load dataset, handle missing values, handle outlier, feature scaling, dan pembagian data menjadi training-testing.

3.1.1 Kode Program preprocessing.py

```
import os
 import pandas as pd
 import numpy as np
 import matplotlib.pyplot as plt
 import seaborn as sns
 from sklearn.model_selection import train_test_split

 def load_and_preprocess():
    # 1. LOAD DATASET
    base_dir = os.path.dirname(os.path.abspath(__file__))
    data_path = os.path.join(base_dir, "data", "water_potability.csv")
    df = pd.read_csv(data_path)
    print(f"Dataset dimuat: {df.shape[0]} baris x {df.shape[1]} kolom")
    print(f"Missing values:\n{df.isnull().sum().to_string()}")

    # 2. HANDLE MISSING VALUES (median tiap kolom)
     df.fillna(df.median(numeric_only=True), inplace=True)

    # 3. DETEKSI & HANDLE OUTLIER dengan IQR method (clip)
    for col in df.select_dtypes(include=np.number).columns:
           if col == "Potability":
           continue
           Q1 = df[col].quantile(0.25)
           Q3 = df[col].quantile(0.75)
           IQR = Q3 - Q1
           lower = Q1 - 1.5 * IQR
           upper = Q3 + 1.5 * IQR
           df[col] = df[col].clip(lower, upper)

    # 4. PISAHKAN FITUR (X) DAN TARGET (y)
    X = df.drop(columns=["Potability"])
    y = df["Potability"]

    # 5. SIMPLE FEATURE SCALING (X / X.max())
    X = X / X.max()

    # 6. SPLIT DATA 80% TRAIN / 20% TEST (stratified)
    X_train, X_test, y_train, y_test = train_test_split(
           X, y, test_size=0.2, stratify=y, random_state=42
    )
    return X_train, X_test, y_train, y_test
```

3.1.2 Penjelasan Kode

1) Load Dataset Menggunakan pandas untuk membaca file water_potability.csv menjadi DataFrame. Jumlah baris (3.276) dan jumlah missing value per kolom ditampilkan di awal untuk verifikasi.

2) Handle Missing Values Metode fillna dengan median dipilih karena median lebih robust terhadap outlier dibandingkan mean. Setiap kolom yang memiliki missing value (ph, Sulfate, Trihalomethanes) diisi dengan nilai tengah dari kolom tersebut.

3) Handle Outlier dengan IQR Untuk setiap fitur numerik (kecuali kolom target Potability), dihitung Interquartile Range (IQR = Q3 – Q1), kemudian batas bawah (Q1 – 1,5*IQR) dan batas atas (Q3 + 1,5*IQR) ditentukan. Semua nilai di luar batas tersebut di-clip (dipaksa ke batas terdekat), sehingga jumlah baris tetap tidak berubah tetapi nilai ekstrem tidak lagi ada.

4) Pisahkan Fitur dan Target Kolom Potability sebagai target (y), sisanya 9 kolom sebagai fitur (X).

5) Simple Feature Scaling Dilakukan dengan membagi tiap fitur dengan nilai maksimalnya (X = X / X.max()). Hasilnya semua nilai fitur berada pada rentang antara 0 sampai 1. Teknik ini dipilih karena sederhana dan sesuai dengan ketentuan soal yang membebaskan pilihan metode transformasi.

6) Split Data Data dibagi menjadi 80% training dan 20% testing menggunakan train_test_split dari scikit-learn. Parameter stratify=y digunakan agar proporsi kelas 0 dan 1 tetap terjaga di data training maupun testing. Parameter random_state=42 memastikan hasil split konsisten setiap kali kode dijalankan.

3.1.3 Output Terminal preprocessing.py

Berikut adalah output di terminal saat file preprocessing.py dijalankan secara mandiri (python preprocessing.py):

Ringkasan output:

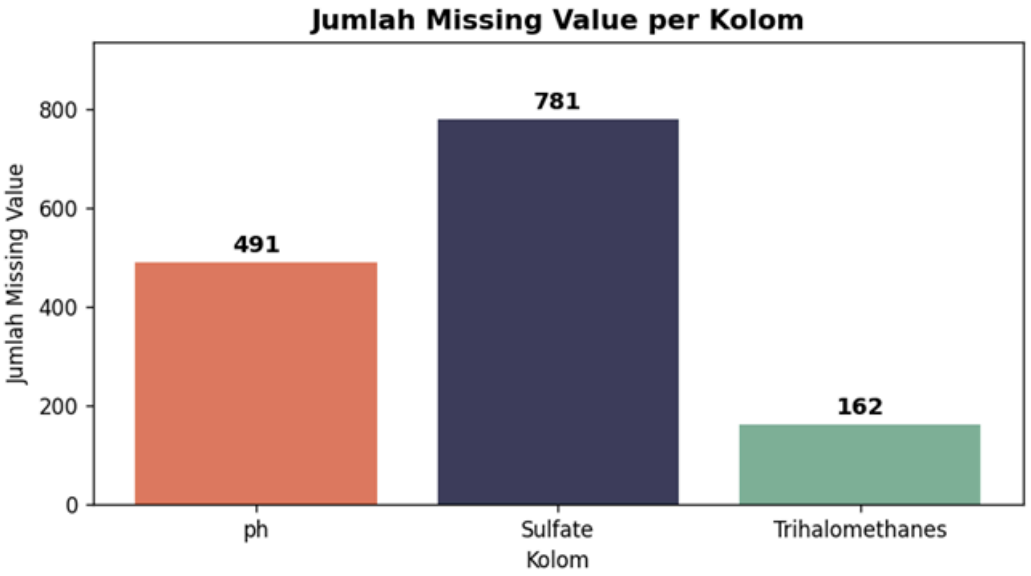
- Dataset dimuat: 3.276 baris x 10 kolom.
- Missing values di-handle: 491 di ph, 781 di Sulfate, 162 di Trihalomethanes → semua terisi median menjadi 0 missing.

- Outlier di-clip: Sulfate (264), ph (142), Hardness (83), Chloramines (61), Trihalomethanes (54), Solids (47), Organic_carbon (25), Turbidity (19), Conductivity (11).
- Feature Scaling berhasil: rentang nilai antara 0,0072 hingga 1,0000.
- Split Data: X_train (2620, 9), X_test (656, 9). y_train {0: 1598, 1: 1022} dan y_test {0: 400, 1: 256}.

3.1.4 Visualisasi Preprocessing

Bagian ini menampilkan empat visualisasi yang dihasilkan dari proses preprocessing:

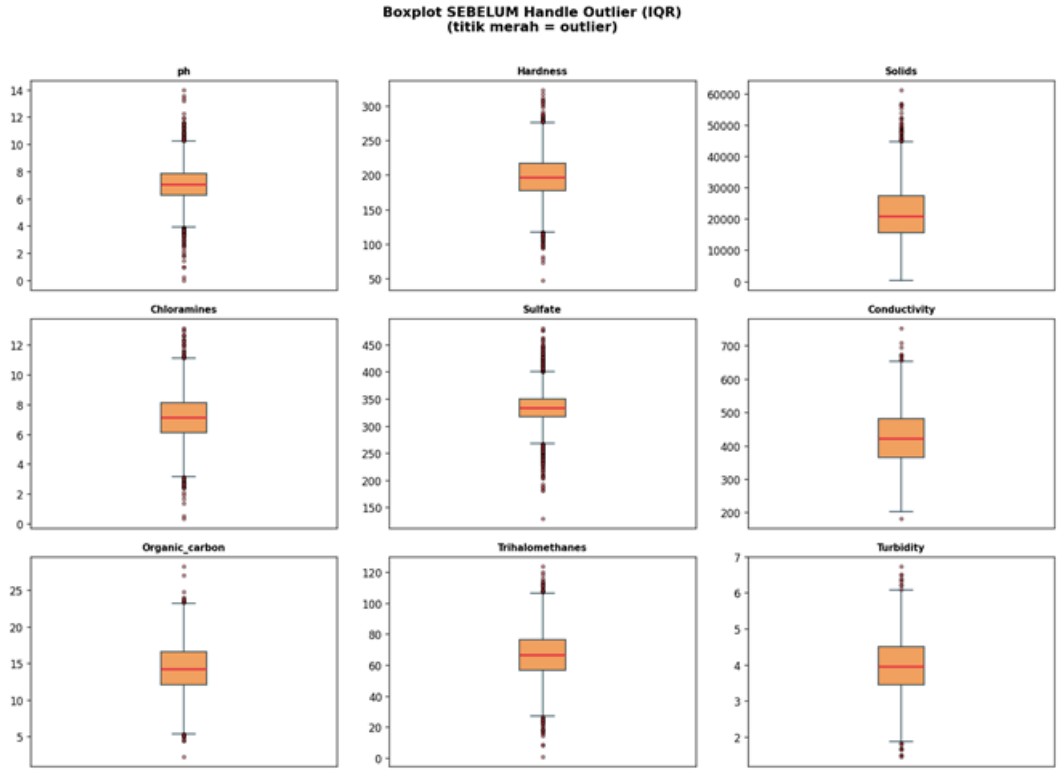
a) Jumlah Missing Value per Kolom



Gambar 3.1 – Jumlah Missing Value per Kolom

Bar chart di atas memperlihatkan jumlah missing value di tiga kolom yang bermasalah. Kolom Sulfate memiliki missing value terbanyak (781), diikuti ph (491), lalu Trihalomethanes (162).

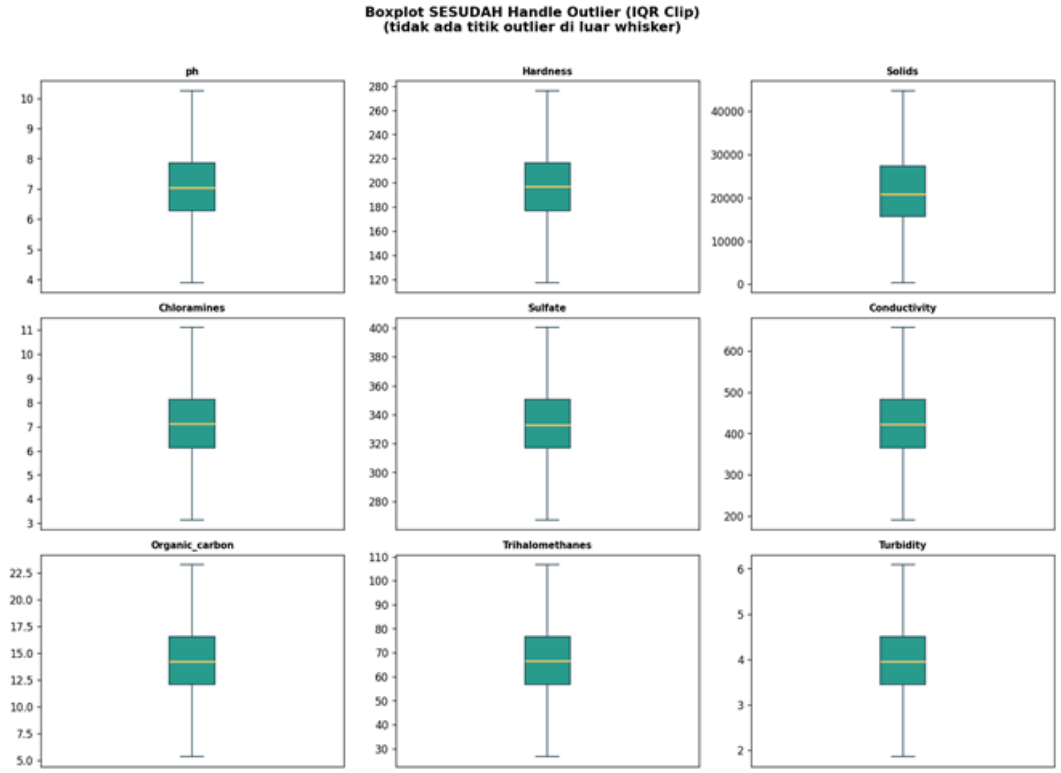
b) Boxplot Sebelum Handle Outlier



Gambar 3.2 – Boxplot Sebelum Handle Outlier (IQR)

Boxplot ini memperlihatkan distribusi tiap fitur sebelum outlier ditangani. Titik-titik merah di luar whisker adalah outlier yang terdeteksi.

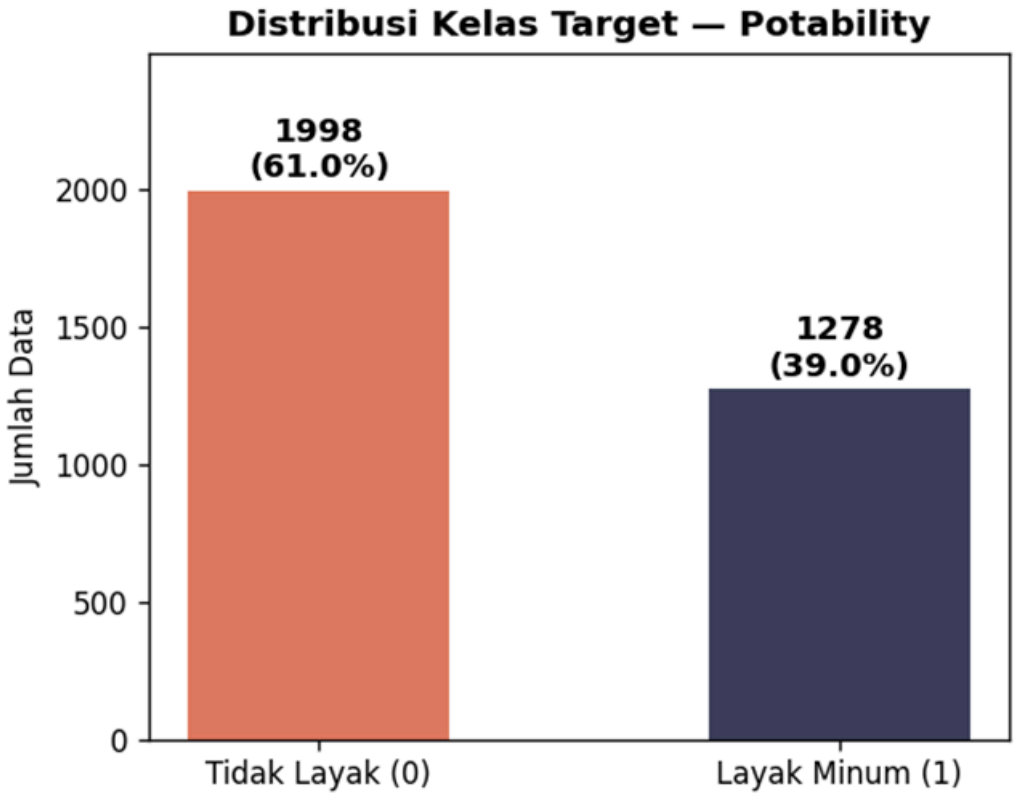
c) Boxplot Sesudah Handle Outlier (IQR Clip)



Gambar 3.3 – Boxplot Sesudah Handle Outlier (IQR Clip)

Setelah metode IQR Clip diterapkan, tidak ada lagi titik outlier di luar whisker. Semua nilai ekstrem telah dibatasi pada batas bawah dan batas atas IQR.

d) Distribusi Kelas Target (Potability)



Gambar 3.4 – Distribusi Kelas Target Potability

Bar chart ini memperlihatkan bahwa dataset tidak seimbang: kelas 0 (Tidak Layak Minum) mendominasi dengan 1.998 sampel (60,99%), sementara kelas 1 (Layak Minum) hanya 1.278 sampel (39,01%). Inilah yang mendasari perlunya resampling pada tahap berikutnya.

3.2 SMOTE + Decision Tree

Bagian ini menangani ketidakseimbangan data training dengan metode SMOTE (Synthetic Minority Oversampling Technique), kemudian melatih model Decision Tree pada data yang sudah seimbang dan mengevaluasi hasilnya pada data testing.

3.2.1 Kode Program smote_pipeline.py

```
import pandas as pd
 import matplotlib.pyplot as plt
 import seaborn as sns
 from imblearn.over_sampling import SMOTE
 from sklearn.tree import DecisionTreeClassifier, plot_tree
 from sklearn.metrics import (confusion_matrix, accuracy_score,
                              precision_score, recall_score, f1_score)

 def run_smote_pipeline(X_train, X_test, y_train, y_test):
    # 1. SMOTE PADA DATA TRAINING
    print("Distribusi sebelum SMOTE:")
    print(y_train.value_counts())

    smote = SMOTE(random_state=42)
    X_train_smote, y_train_smote = smote.fit_resample(X_train, y_train)

    print("Distribusi setelah SMOTE:")
    print(y_train_smote.value_counts())

    # 2. TRAIN DECISION TREE
    model = DecisionTreeClassifier(random_state=42)
    model.fit(X_train_smote, y_train_smote)

    # 3. PREDIKSI DATA TESTING
    y_pred = model.predict(X_test)

    # 4. EVALUASI: Confusion Matrix + Metrik
    cm   = confusion_matrix(y_test, y_pred)
    acc  = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred, zero_division=0)
    rec  = recall_score(y_test, y_pred, zero_division=0)
    f1   = f1_score(y_test, y_pred, zero_division=0)

    print("HASIL EVALUASI")
    print("Confusion Matrix:\n", cm)
    print(f"Accuracy : {acc:.4f}")
    print(f"Precision: {prec:.4f}")
    print(f"Recall   : {rec:.4f}")
    print(f"F1-Score : {f1:.4f}")

    return {"model": model, "confusion_matrix": cm,
           "accuracy": acc, "precision": prec,
           "recall": rec, "f1_score": f1}
```

3.2.2 Penjelasan Kode

1) SMOTE pada Data Training SMOTE bekerja dengan cara membuat sampel sintetis baru untuk kelas minoritas (Potability = 1). Sampel sintetis ini dihasilkan dengan menginterpolasi antar nilai fitur dari beberapa sampel kelas minoritas yang berdekatan (menggunakan K-Nearest Neighbors). Hasilnya: kelas minoritas dinaikkan jumlahnya hingga sama dengan kelas mayoritas. Penting: SMOTE hanya dilakukan pada data training, tidak pada data testing, agar hasil evaluasi tetap merepresentasikan kondisi dunia nyata yang imbalanced.

2) Train Decision Tree Model DecisionTreeClassifier dari sklearn digunakan dengan parameter default dan random_state=42 (untuk reproducibility). Model dilatih pada data training yang sudah di-SMOTE (1598 sampel kelas 0 dan 1598 sampel kelas 1).

3) Prediksi Data Testing Model yang sudah dilatih digunakan untuk memprediksi kelas dari 656 data testing (yang tidak di-resample).

4) Evaluasi Confusion matrix dihitung untuk menyusun jumlah True Negative, False Positive, False Negative, dan True Positive. Empat metrik utama (Accuracy, Precision, Recall, F1-Score) kemudian dihitung untuk kelas positif (Potability = 1).

3.2.3 Output Terminal smote_pipeline.py

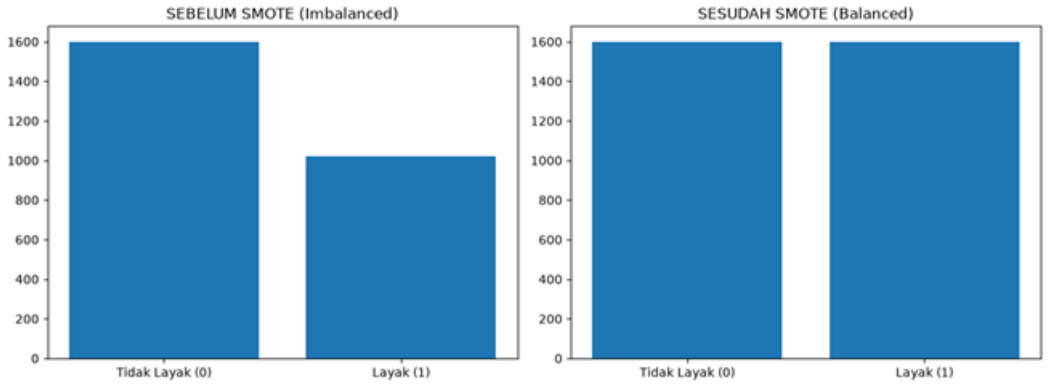
Ringkasan hasil:

- Distribusi sebelum SMOTE: {0: 1598, 1: 1022} – imbalanced.
- Distribusi setelah SMOTE: {0: 1598, 1: 1598} – seimbang.
- Confusion Matrix: [[240, 160], [126, 130]] → TN=240, FP=160, FN=126, TP=130.
- Accuracy = 0,5640 (56,40%), Precision = 0,4483 (44,83%), Recall = 0,5078 (50,78%), F1-Score = 0,4762 (47,62%).

3.2.4 Visualisasi SMOTE

Berikut visualisasi yang dihasilkan dari pipeline SMOTE:

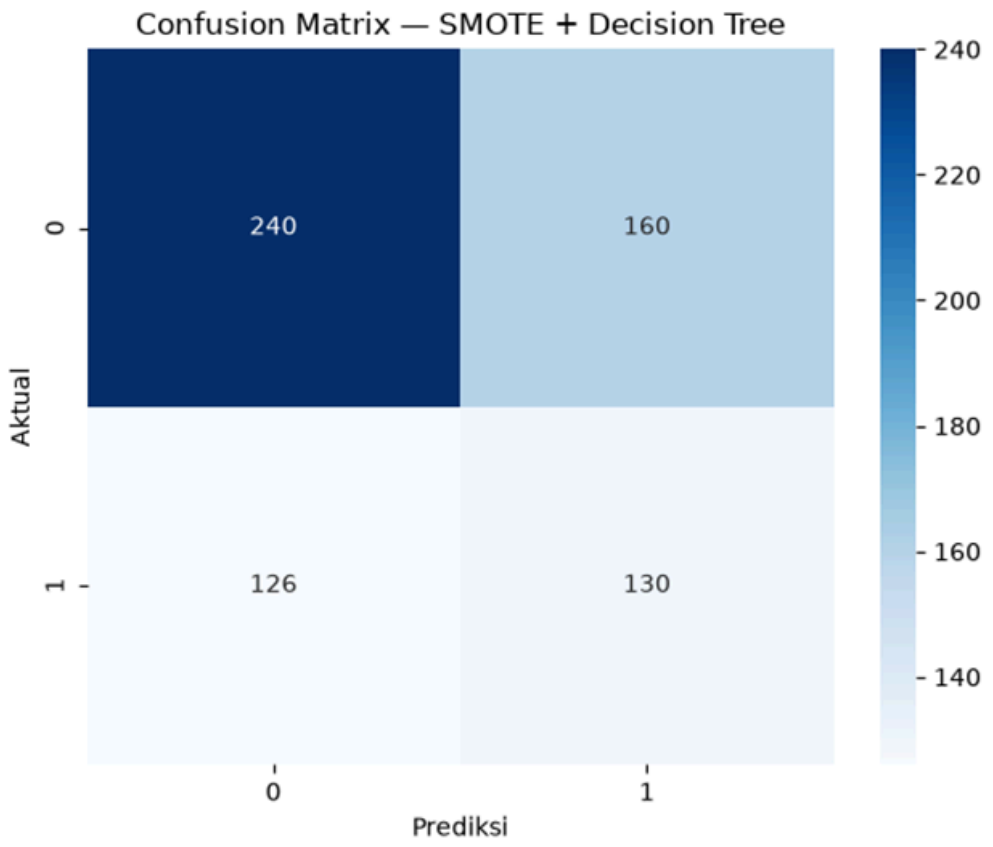
a) Distribusi Kelas y_train Sebelum vs Sesudah SMOTE



Gambar 3.5 – Distribusi Kelas Sebelum dan Sesudah SMOTE

Bar chart membandingkan distribusi kelas sebelum dan sesudah SMOTE. Terlihat SMOTE menambah 576 sampel sintetis pada kelas minoritas hingga sama jumlahnya dengan kelas mayoritas (1598).

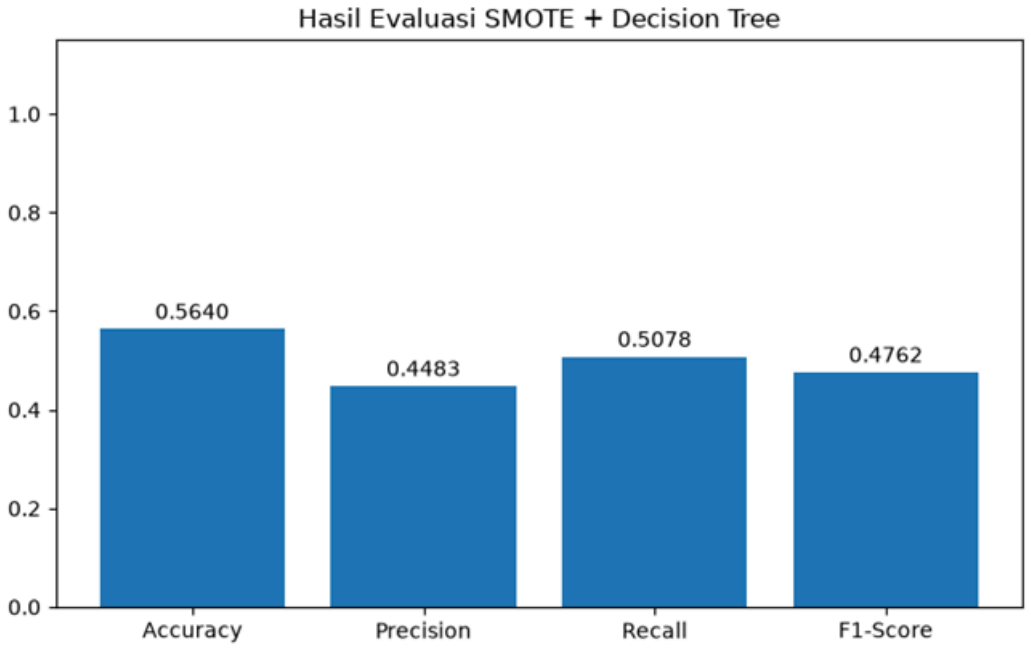
b) Confusion Matrix Heatmap



Gambar 3.6 – Confusion Matrix SMOTE

Heatmap Confusion Matrix menampilkan hasil klasifikasi: 240 True Negative, 160 False Positive, 126 False Negative, dan 130 True Positive.

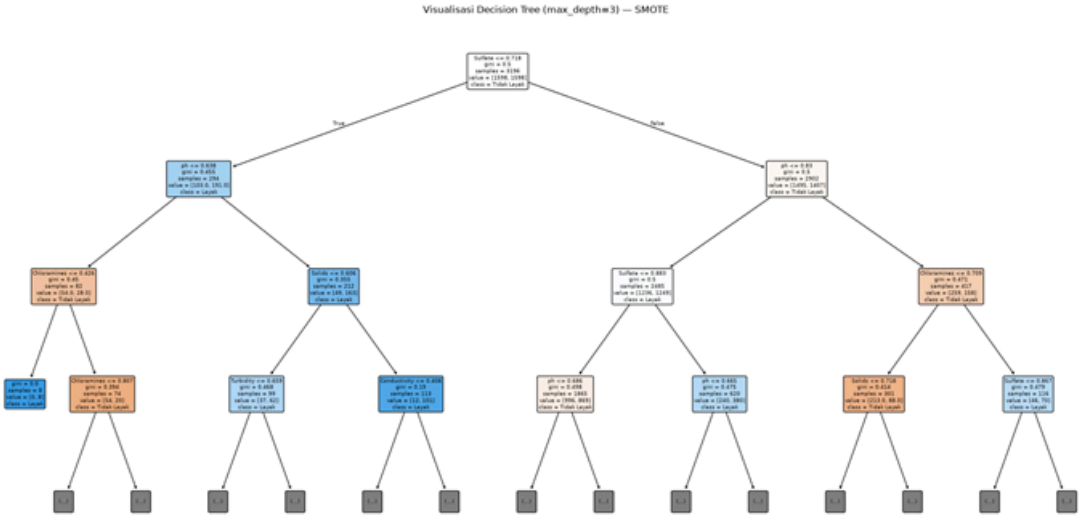
c) Bar Chart Metrik Evaluasi



Gambar 3.7 – Metrik Evaluasi SMOTE

Bar chart menampilkan keempat metrik evaluasi. Accuracy dan Recall terlihat lebih tinggi dibanding Precision, menunjukkan model condong memprediksi kelas positif meski tidak selalu benar.

d) Visualisasi Decision Tree



Gambar 3.8 – Visualisasi Decision Tree (SMOTE, max_depth=3)

Visualisasi struktur Decision Tree yang dilatih pada data SMOTE (dibatasi kedalamannya max_depth=3 agar mudah dibaca).

3.3 RUS + Decision Tree

Bagian ini menangani ketidakseimbangan data dengan metode Random Under Sampling (RUS), kemudian melatih Decision Tree pada data yang sudah seimbang dan mengevaluasi hasilnya.

3.3.1 Kode Program rus_pipeline.py

```
import pandas as pd
 import matplotlib.pyplot as plt
 import seaborn as sns
 from imblearn.under_sampling import RandomUnderSampler
 from sklearn.tree import DecisionTreeClassifier, plot_tree
 from sklearn.metrics import (confusion_matrix, accuracy_score,
                              precision_score, recall_score, f1_score,
                              classification_report)

 def run_rus_pipeline(X_train, X_test, y_train, y_test):
    # 1. RANDOM UNDER SAMPLING pada data training
    print("Distribusi y_train SEBELUM RUS:")
     print(dict(pd.Series(y_train).value_counts().sort_index()))

    rus = RandomUnderSampler(random_state=42)
    X_train_rus, y_train_rus = rus.fit_resample(X_train, y_train)

    print("Distribusi y_train SESUDAH RUS:")
     print(dict(pd.Series(y_train_rus).value_counts().sort_index()))

    # 2. TRAIN DECISION TREE pada data yang sudah di-RUS
    decision_tree = DecisionTreeClassifier(random_state=42)
    decision_tree.fit(X_train_rus, y_train_rus)

    # 3. PREDIKSI pada data testing (TIDAK di-resample)
    y_pred = decision_tree.predict(X_test)

    # 4. EVALUASI: Confusion Matrix + Metrik
    cm   = confusion_matrix(y_test, y_pred)
    acc  = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred)
    rec  = recall_score(y_test, y_pred)
    f1   = f1_score(y_test, y_pred)

    print("Hasil Evaluasi:")
    print(f"Accuracy  : {acc:.4f}")
    print(f"Precision : {prec:.4f}")
    print(f"Recall      : {rec:.4f}")
    print(f"F1-Score  : {f1:.4f}")

    return {"model": decision_tree, "confusion_matrix": cm,
           "accuracy": acc, "precision": prec,
           "recall": rec, "f1_score": f1}
```

3.3.2 Penjelasan Kode

1) Random Under Sampling Berbeda dengan SMOTE yang menambah sampel minoritas, RUS bekerja dengan cara membuang sampel dari kelas mayoritas secara acak hingga jumlahnya sama dengan kelas minoritas. Dalam kasus ini, 576 sampel kelas 0 dibuang, sehingga baik kelas 0 maupun kelas 1 sama-sama berjumlah 1022 sampel.

Kelebihan RUS adalah cepat dan sederhana; kekurangannya adalah informasi dari kelas mayoritas ada yang hilang.

2) Train Decision Tree Sama seperti SMOTE, model DecisionTreeClassifier default digunakan dengan random_state=42. Konsistensi parameter ini memungkinkan perbandingan yang adil antara kedua metode.

3) Prediksi Data Testing Model memprediksi 656 data testing yang tidak diresample.

4) Evaluasi Confusion matrix dan empat metrik (Accuracy, Precision, Recall, F1-Score) dihitung. Selain itu ditampilkan juga classification_report untuk melihat metrik per kelas.

3.3.3 Output Terminal rus_pipeline.py

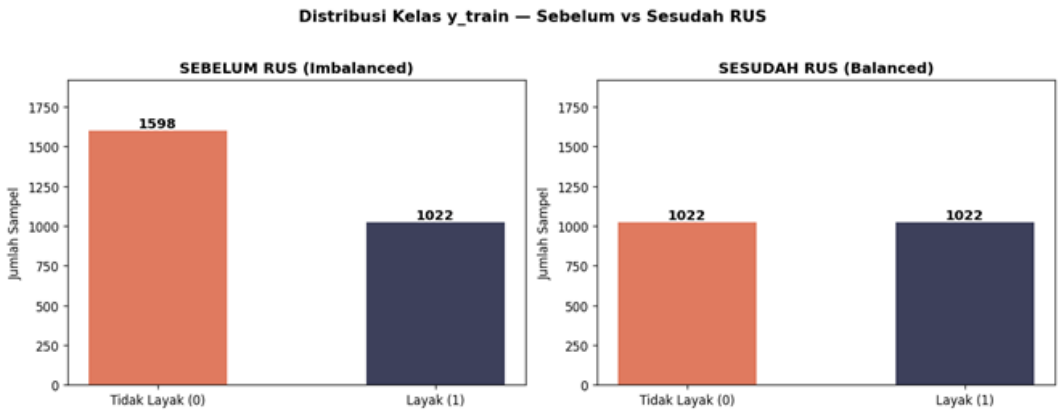
Ringkasan hasil:

- Distribusi y_train SEBELUM RUS: {0: 1598, 1: 1022} – imbalanced.
- Distribusi y_train SESUDAH RUS: {0: 1022, 1: 1022} – seimbang.
- Confusion Matrix: [[227, 173], [122, 134]] → TN=227, FP=173, FN=122, TP=134.
- Accuracy = 0,5503 (55,03%), Precision = 0,4365 (43,65%), Recall = 0,5234 (52,34%), F1-Score = 0,4760 (47,60%).

3.3.4 Visualisasi RUS

Berikut visualisasi yang dihasilkan dari pipeline RUS:

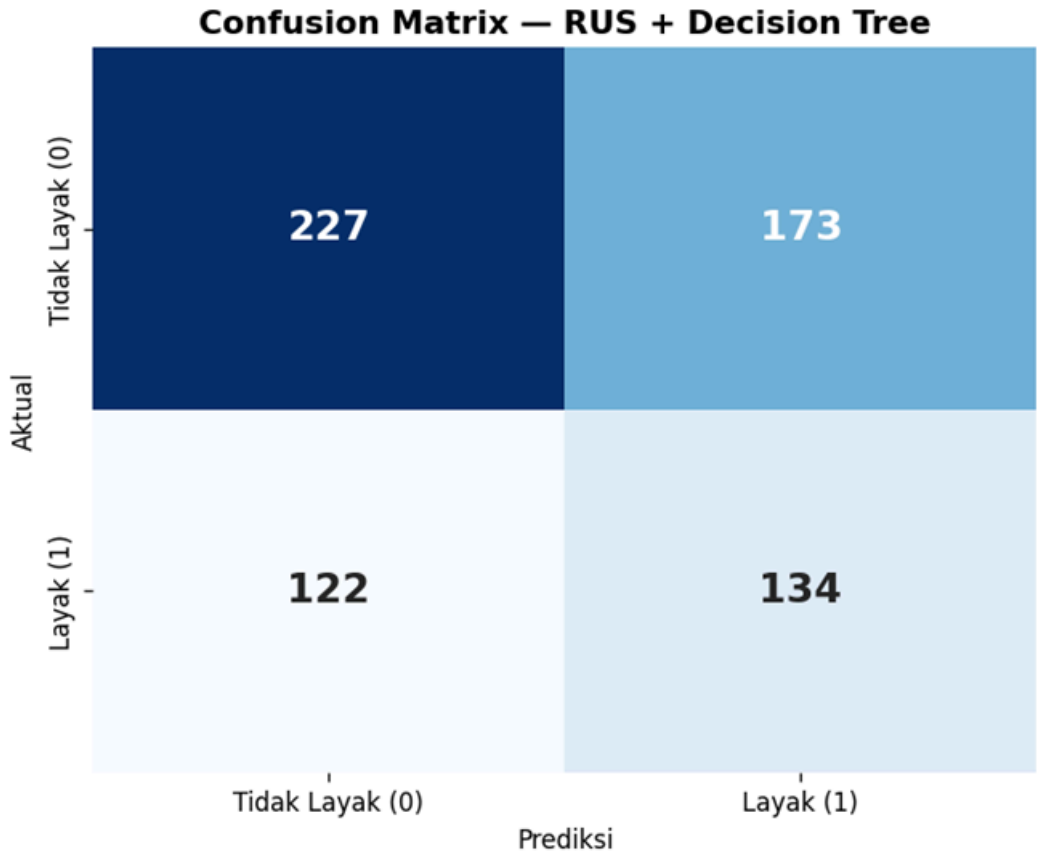
a) Distribusi Kelas y_train Sebelum vs Sesudah RUS



Gambar 3.9 – Distribusi Kelas Sebelum dan Sesudah RUS

Bar chart membandingkan distribusi kelas sebelum dan sesudah RUS. Terlihat RUS membuang 576 sampel dari kelas mayoritas sehingga kedua kelas sama-sama 1022 sampel.

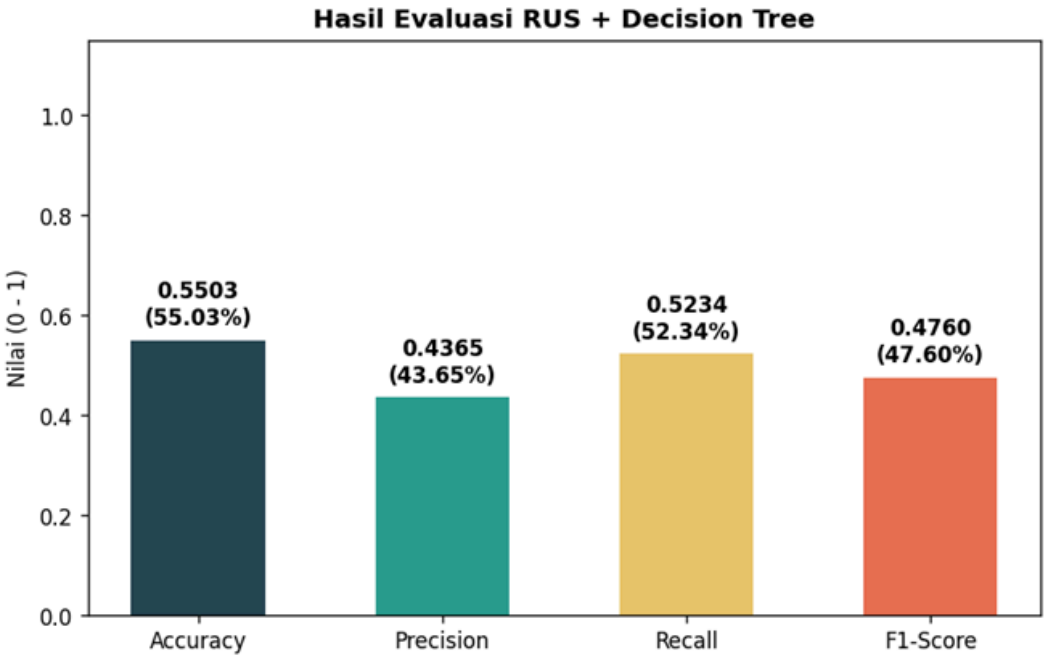
b) Confusion Matrix Heatmap



Gambar 3.10 – Confusion Matrix RUS

Heatmap Confusion Matrix menampilkan: 227 True Negative, 173 False Positive, 122 False Negative, dan 134 True Positive.

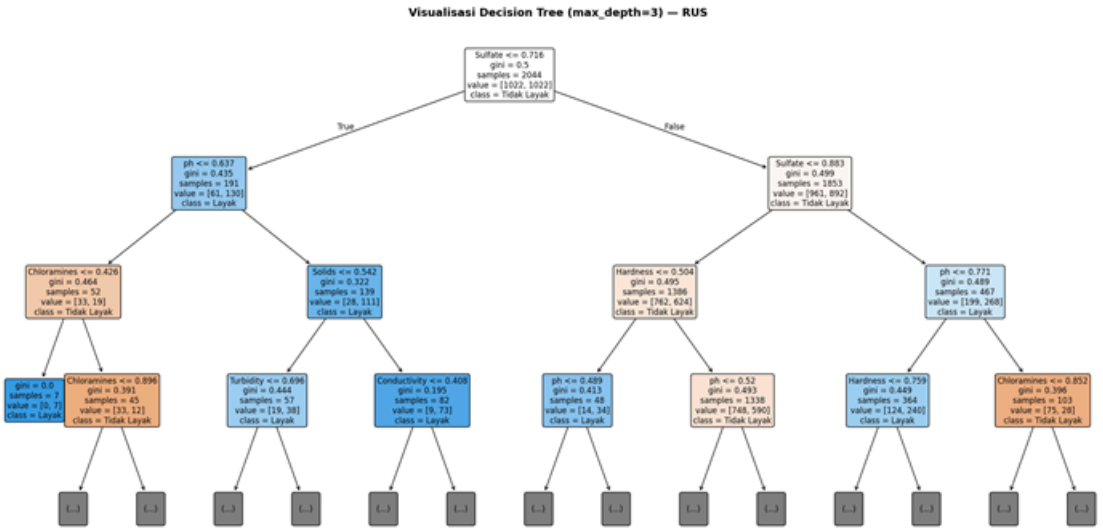
c) Bar Chart Metrik Evaluasi



Gambar 3.11 – Metrik Evaluasi RUS

Bar chart menampilkan keempat metrik evaluasi. Nilai Recall (52,34%) sedikit lebih tinggi dari Accuracy (55,03%) dan Precision (43,65%), menunjukkan RUS cukup baik dalam menangkap kelas minoritas.

d) Visualisasi Decision Tree



Gambar 3.12 – Visualisasi Decision Tree (RUS, max_depth=3)

Visualisasi struktur Decision Tree yang dilatih pada data RUS (dibatasi kedalamannya max_depth=3).

3.3.5 Kode Program main.py

File main.py adalah penghubung yang memanggil ketiga bagian secara berurutan:

```
from preprocessing import load_and_preprocess
 from smote_pipeline import run_smote_pipeline
 from rus_pipeline import run_rus_pipeline

 if __name__ == "__main__":
    # Step 1: Preprocessing & Split Data (Adel)
    X_train, X_test, y_train, y_test = load_and_preprocess()

    # Step 2: SMOTE + Decision Tree (Juanne)
    run_smote_pipeline(X_train, X_test, y_train, y_test)

    # Step 3: RUS + Decision Tree (Risha)
    run_rus_pipeline(X_train, X_test, y_train, y_test)
```

Ketika dijalankan (python main.py), ketiga bagian akan berjalan secara berurutan: preprocessing → SMOTE pipeline → RUS pipeline. Hasil kedua metode resampling akan tampil berurutan di terminal untuk memudahkan perbandingan.

BAB 4
PERBANDINGAN DAN ANALISIS

Bab ini membandingkan performa metode SMOTE dan RUS dari sisi metrik evaluasi, membedah confusion matrix keduanya, serta memberikan interpretasi terhadap perbedaan hasil.

4.1 Tabel Perbandingan Metrik

Berikut adalah rangkuman hasil evaluasi kedua metode pada data testing (656 sampel, yang tidak di-resample):

Metrik	SMOTE	RUS	Pemenang
Accuracy	56,40%	55,03%	SMOTE (+1,37%)
Precision	44,83%	43,65%	SMOTE (+1,18%)
Recall	50,78%	52,34%	RUS (+1,56%)
F1-Score	47,62%	47,60%	Setara (~ seri)

4.2 Analisis Confusion Matrix

Confusion matrix menampilkan detail jenis kesalahan yang dilakukan oleh masing-masing model:

Metode	TN	FP	FN	TP
SMOTE	240	160	126	130
RUS	227	173	122	134

Keterangan: TN = benar mengklasifikasi air tidak layak, FP = salah mengklasifikasi air tidak layak sebagai layak, FN = salah mengklasifikasi air layak sebagai tidak layak, TP = benar mengklasifikasi air layak.

Dari tabel di atas dapat dilihat bahwa SMOTE menghasilkan TN lebih tinggi (240 vs 227) dan FP lebih rendah (160 vs 173), sehingga SMOTE lebih baik dalam mengidentifikasi air yang tidak layak minum dengan benar. Sedangkan RUS menghasilkan TP lebih tinggi (134 vs 130) dan FN lebih rendah (122 vs 126), artinya RUS lebih baik dalam mengidentifikasi air yang layak minum tanpa terlewat.

4.3 Interpretasi Hasil

1) SMOTE unggul di Accuracy dan Precision. Hal ini dapat dijelaskan karena SMOTE tidak membuang data mayoritas, malah menambah data sintetis untuk kelas

minoritas. Model punya lebih banyak informasi untuk belajar dari kelas mayoritas, sehingga saat memprediksi kelas positif (Layak Minum), tingkat kesalahannya (False Positive) lebih rendah. Precision yang lebih tinggi berarti ketika model berkata 'ini layak minum', kemungkinan besar memang benar layak.

2) RUS unggul di Recall. RUS membuang sampel kelas mayoritas sehingga model menjadi lebih sensitif terhadap kelas minoritas. Model tidak lagi bias ke kelas 0 karena jumlah kedua kelas seimbang, dan tidak ada tambahan informasi sintetis yang bisa mengecoh pembelajaran kelas 1. Akibatnya, model RUS lebih jarang melewatkan air yang sebenarnya layak minum (False Negative lebih rendah), sehingga Recall lebih tinggi.

3) F1-Score keduanya hampir identik. F1-Score adalah harmonic mean antara Precision dan Recall. Karena SMOTE unggul di Precision dan RUS unggul di Recall (dengan selisih yang mirip), maka F1-Score keduanya menjadi hampir sama (47,62% vs 47,60%). Ini menandakan kedua metode memiliki trade-off yang seimbang antara mengenali kelas positif dan tidak salah mengklasifikasi kelas negatif.

4.4 Rekomendasi Metode

Pemilihan metode terbaik bergantung pada konteks aplikasi:

- Jika prioritasnya adalah kepercayaan hasil positif (ketika model bilang 'layak minum', harus benar), maka SMOTE lebih tepat karena Precision-nya lebih tinggi.
- Jika prioritasnya adalah tidak melewatkan air yang benar-benar layak minum (menghindari False Negative), maka RUS lebih tepat karena Recall-nya lebih tinggi.

Untuk konteks kualitas air minum yang menyangkut kesehatan, umumnya lebih penting untuk tidak salah menyebutkan air 'tidak layak' padahal layak (menghindari FN), sehingga RUS bisa dinilai lebih relevan secara praktis. Namun, karena angka akurasi keseluruhan SMOTE lebih tinggi, jika sumber daya komputasi tidak terbatas, SMOTE tetap menjadi pilihan yang lebih umum digunakan pada dataset imbalanced.

BAB 5
KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan implementasi dan analisis yang telah dilakukan, dapat disimpulkan beberapa hal sebagai berikut:

1) Dataset Water Potability dari Kaggle merupakan dataset imbalanced dengan proporsi 60,99% kelas 0 dan 39,01% kelas 1, memiliki missing values pada tiga kolom (ph, Sulfate, Trihalomethanes), dan sejumlah outlier pada seluruh fitur numeriknya.
2) Preprocessing yang dilakukan meliputi handle missing value dengan median, handle outlier dengan metode IQR clip, Simple Feature Scaling (X/X.max), dan split data 80:20 dengan stratified sampling untuk menjaga proporsi kelas.
3) SMOTE menghasilkan data training seimbang dengan menambah sampel sintetis untuk kelas minoritas (1598:1022 menjadi 1598:1598), sedangkan RUS menghasilkan data training seimbang dengan membuang sampel kelas mayoritas (1598:1022 menjadi 1022:1022).
4) Hasil evaluasi menggunakan Decision Tree menunjukkan performa yang mirip antara kedua metode: SMOTE mencapai Accuracy 56,40%, Precision 44,83%, Recall 50,78%, dan F1-Score 47,62%; sedangkan RUS mencapai Accuracy 55,03%, Precision 43,65%, Recall 52,34%, dan F1-Score 47,60%.
5) SMOTE lebih unggul di Accuracy dan Precision, sementara RUS lebih unggul di Recall. F1-Score keduanya hampir identik, menunjukkan trade-off precision-recall yang seimbang antara kedua metode.
6) Pemilihan metode terbaik bergantung pada konteks penggunaan: SMOTE cocok jika prioritasnya adalah akurasi umum dan kepercayaan prediksi positif, sedangkan RUS cocok jika prioritasnya adalah tidak melewatkan kelas minoritas.

5.2 Saran

Untuk penelitian lanjutan, beberapa saran yang dapat dilakukan:

1) Melakukan tuning parameter Decision Tree seperti max_depth, min_samples_split, dan criterion untuk mencari kombinasi terbaik.
2) Mencoba algoritma klasifikasi lain seperti Random Forest, Gradient Boosting, atau XGBoost yang biasanya menghasilkan performa lebih tinggi pada data tabular.

3) Menggunakan teknik resampling hybrid seperti SMOTE-Tomek atau SMOTE-ENN yang menggabungkan oversampling dan undersampling.
4) Melakukan feature engineering tambahan seperti membuat fitur interaksi antar kolom atau membuang fitur yang kurang informatif berdasarkan feature importance.
5) Menggunakan cross-validation untuk menilai stabilitas performa model, bukan hanya split data tunggal.